

AIAC LAB 8.1

Name : K Srishanth
Roll No : 2503A52701
Batch No : 24

Task Description #1 – Password Strength Validator

Question:

Apply AI to generate at least 3 assert test cases for `is_strong_password(password)` and implement the validator function.

Answer:

The Python program below implements the required function/class and verifies it using assert-based test cases.

#code

```
expl.py • Python
1  def is_strong_password(password):
2      if len(password) < 8 or " " in password:
3          return False
4
5      has_upper = any(c.isupper() for c in password)
6      has_lower = any(c.islower() for c in password)
7      has_digit = any(c.isdigit() for c in password)
8      has_special = any(not c.isalnum() for c in password)
9
10     return has_upper and has_lower and has_digit and has_special
11
12
13     # Assert test cases
14     assert is_strong_password("Abcd@123") == True
15     assert is_strong_password("abcd123") == False
16     assert is_strong_password("Abcd@1234") == True
17
18     print("All password validation tests passed")
19
20     # User input
21     password = input("Enter a password: ")
22
23     if is_strong_password(password):
24         print("Result: Strong Password")
25     else:
26         print("Result: Weak Password")
```

#output

```
TERMINAL
PS C:\Users\V\OneDrive\AIAC> python expl.py
All password validation tests passed
Enter a password: Abcd@123
Result: Strong Password
PS C:\Users\V\OneDrive\AIAC> python expl.py
All password validation tests passed
Enter a password: abcd123
Result: Weak Password
```

Task Description #2 – Number Classification with Loops

Question:

Use AI to generate at least 3 assert test cases for a `classify_number(n)` function and implement it using loops.

Answer:

The Python program below implements the required function/class and verifies it using assert-based test cases.

#code

```
exp2.py • Python

1  def classify_number(n):
2      if not isinstance(n, (int, float)):
3          return "Invalid"
4
5      for _ in range(1):
6          if n > 0:
7              return "Positive"
8          elif n < 0:
9              return "Negative"
10         else:
11             return "Zero"
12
13
14  # Assert test cases
15  assert classify_number(10) == "Positive"
16  assert classify_number(-5) == "Negative"
17  assert classify_number(0) == "Zero"
18  assert classify_number(1) == "Positive"
19  assert classify_number(-1) == "Negative"
20  assert classify_number("10") == "Invalid"
21  assert classify_number(None) == "Invalid"
22
23  print("All number classification tests passed")
24
25  # User input (loop runs until the user types 'q')
26  while True:
27      value = input("Enter a number (or 'q' to quit): ")
28
29      if value.lower() == "q":
30          break
31
32      try:
33          number = int(value)
34      except ValueError:
35          try:
36              number = float(value)
37          except ValueError:
38              number = value
39
40      print("Result:", classify_number(number))
41
42  print("Program ended")
```

#output

TERMINAL

```
PS C:\Users\V\OneDrive\AIAC> python exp2.py
All number classification tests passed
Enter a number (or 'q' to quit): 10
Result: Positive
Enter a number (or 'q' to quit): -5
Result: Negative
Enter a number (or 'q' to quit): 0
Result: Zero
Enter a number (or 'q' to quit): abc
Result: Invalid
Enter a number (or 'q' to quit): q
Program ended
```

Task Description #3 – Anagram Checker

Question:

Use AI to generate at least 3 assert test cases for `is_anagram(str1, str2)` and implement the function.

Answer:

The Python program below implements the required function/class and verifies it using assert-based test cases.

#code

```
exp3.py • Python
1  def normalize(text):
2      return "".join(c.lower() for c in text if c.isalnum())
3
4
5  def is_anagram(str1, str2):
6      return sorted(normalize(str1)) == sorted(normalize(str2))
7
8
9  # Assert test cases
10 assert is_anagram("listen", "silent") == True
11 assert is_anagram("hello", "world") == False
12 assert is_anagram("Dormitory", "Dirty Room") == True
13 assert is_anagram("", "") == True
14 assert is_anagram("School Master!", "The Classroom") == True
15
16 print("All anagram tests passed")
17
18 # User input
19 str1 = input("Enter first string: ")
20 str2 = input("Enter second string: ")
21
22 if is_anagram(str1, str2):
23     print("Result: The strings are anagrams")
24 else:
25     print("Result: The strings are not anagrams")
```

#output

```
TERMINAL
PS C:\Users\V\OneDrive\AIAC> python exp3.py
All anagram tests passed
Enter first string: listen
Enter second string: silent
Result: The strings are anagrams
PS C:\Users\V\OneDrive\AIAC> python exp3.py
All anagram tests passed
Enter first string: hello
Enter second string: world
Result: The strings are not anagrams
```

Task Description #4 – Inventory Class

Question:

Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

Answer:

The Python program below implements the required function/class and verifies it using assert-based test cases.

#code

```
exp4.py • Python
1  class Inventory:
2      def __init__(self):
3          self.stock = {}
4
5      def add_item(self, name, quantity):
6          self.stock[name] = self.stock.get(name, 0) + quantity
7
8      def remove_item(self, name, quantity):
9          if self.stock.get(name, 0) < quantity:
10             return False
11             self.stock[name] -= quantity
12             return True
13
14      def get_stock(self, name):
15          return self.stock.get(name, 0)
16
17
18  # Assert test cases
19  test_inv = Inventory()
20  test_inv.add_item("Pen", 10)
21  assert test_inv.get_stock("Pen") == 10
22  test_inv.remove_item("Pen", 5)
23  assert test_inv.get_stock("Pen") == 5
24  test_inv.add_item("Book", 3)
25  assert test_inv.get_stock("Book") == 3
26  assert test_inv.remove_item("Book", 10) == False
27
28  print("All inventory tests passed")
29
30  # User input (menu driven)
31  inv = Inventory()
32
33  while True:
34      print("\n1. Add item  2. Remove item  3. Check stock  4. Exit")
35      choice = input("Enter your choice: ")
36
37      if choice == "1":
38          name = input("Enter item name: ")
39          qty = int(input("Enter quantity: "))
40          inv.add_item(name, qty)
41          print("Added", qty, name)
42      elif choice == "2":
43          name = input("Enter item name: ")
44          qty = int(input("Enter quantity: "))
45          if inv.remove_item(name, qty):
46              print("Removed", qty, name)
47          else:
48              print("Not enough stock")
49      elif choice == "3":
50          name = input("Enter item name: ")
51          print(name, "stock =", inv.get_stock(name))
52      elif choice == "4":
53          print("Exiting inventory")
54          break
55      else:
56          print("Invalid choice")
```

#output

TERMINAL

```
PS C:\Users\V\OneDrive\AIAC> python exp4.py  
All inventory tests passed
```

```
1. Add item 2. Remove item 3. Check stock 4. Exit  
Enter your choice: 1  
Enter item name: Pen  
Enter quantity: 10  
Added 10 Pen
```

```
1. Add item 2. Remove item 3. Check stock 4. Exit  
Enter your choice: 2  
Enter item name: Pen  
Enter quantity: 5  
Removed 5 Pen
```

```
1. Add item 2. Remove item 3. Check stock 4. Exit  
Enter your choice: 1  
Enter item name: Book  
Enter quantity: 3  
Added 3 Book
```

```
1. Add item 2. Remove item 3. Check stock 4. Exit  
Enter your choice: 3  
Enter item name: Pen  
Pen stock = 5
```

```
1. Add item 2. Remove item 3. Check stock 4. Exit  
Enter your choice: 2  
Enter item name: Pen  
Enter quantity: 20  
Not enough stock
```

```
1. Add item 2. Remove item 3. Check stock 4. Exit  
Enter your choice: 4  
Exiting inventory
```

Task Description #5 – Date Validation & Formatting

Question:

Use AI to generate at least 3 assert test cases for `validate_and_format_date(date_str)` to check and convert dates.

Answer:

The Python program below implements the required function/class and verifies it using assert-based test cases.

#code

```
exp5.py • Python
1  from datetime import datetime
2
3
4  def validate_and_format_date(date_str):
5      try:
6          date = datetime.strptime(date_str, "%m/%d/%Y")
7          return date.strftime("%Y-%m-%d")
8      except ValueError:
9          return "Invalid Date"
10
11
12  # Assert test cases
13  assert validate_and_format_date("10/15/2023") == "2023-10-15"
14  assert validate_and_format_date("02/30/2023") == "Invalid Date"
15  assert validate_and_format_date("01/01/2024") == "2024-01-01"
16  assert validate_and_format_date("13/01/2024") == "Invalid Date"
17
18  print("All date validation tests passed")
19
20  # User input
21  date_str = input("Enter date (MM/DD/YYYY): ")
22
23  print("Formatted Date:", validate_and_format_date(date_str))
```

#output

```
TERMINAL
PS C:\Users\V\OneDrive\AIAC> python exp5.py
All date validation tests passed
Enter date (MM/DD/YYYY): 10/15/2023
Formatted Date: 2023-10-15
PS C:\Users\V\OneDrive\AIAC> python exp5.py
All date validation tests passed
Enter date (MM/DD/YYYY): 02/30/2023
Formatted Date: Invalid Date
```

Conclusion

All five tasks were implemented and verified using assert-based test cases. The programs cover validation, edge cases, and expected outputs as required in Lab 8.